Voya FYP Report — Implementation, Deployment & Conclusion

Oct 25, 2026 · @yy

Chapter 5: Implementation and Testing

Voya, the AI-Driven Contextual Travel Management system, was implemented in the third increment of the incremental model described in Section 1.6.1. It consists of a Flutter mobile app for travellers (about 43,400 lines of Dart), a Firebase backend with four Cloud Functions, a web console for administrators and insurance partners, and a self-trained discussion-summary model. This chapter explains how each module from Chapter 4 was built and how the system was tested.

5.1 Implementation

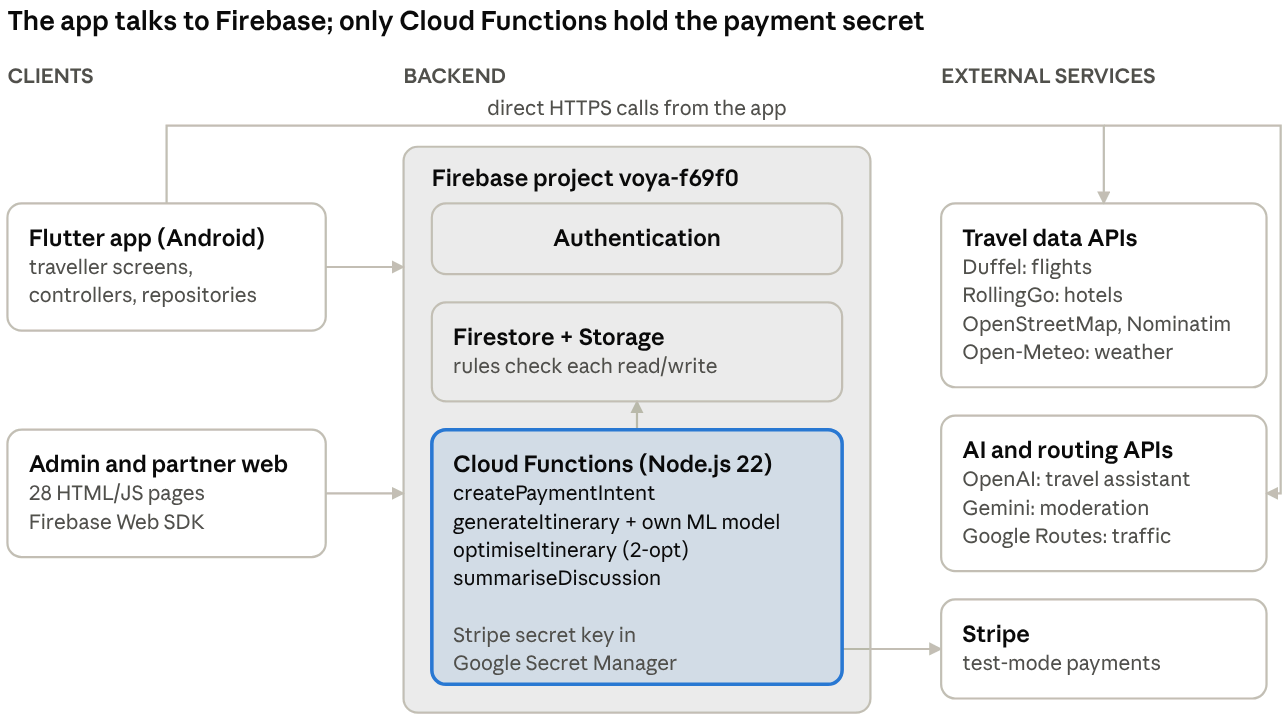


Figure 5.1 · Voya system architecture

Both clients use the Firebase SDK for login, data and function calls. The app calls the travel data and AI APIs directly, while payment goes only through a Cloud Function that holds the Stripe key.

5.1.1 Development tools and technologies

Layer	Technology used	Planned in Chapter 2	Purpose in Voya
Mobile client	Flutter (Dart SDK ^3.10), Android Studio	Same (2.1.5)	Cross-platform UI for travellers
Web consoles	HTML, CSS, JavaScript + Firebase Web SDK	Same (2.2.1)	Admin and insurance-partner portals
Authentication	Firebase Authentication	Same	Email/password and Google sign-in, password reset
Database and files	Cloud Firestore, Firebase Storage	Same (2.1.6)	Real-time storage for users, trips, chats, votes, expenses, catalog, receipts
Server logic	Cloud Functions (Node.js 22)	Node.js + Python Flask	Payment, AI Summarise, optimise, discussion summary; no separate Flask server was needed
Maps and places	OpenStreetMap / Nominatim, `flutter_map`, Google Routes API (traffic only)	Google Maps and Google Places	Place data, geocoding, maps, traffic-aware travel time
Payments	Stripe (test mode), `flutter_stripe`	PayPal	Flight, hotel and insurance checkout
Travel data	Duffel API (flights), RollingGo (hotels), Open-Meteo (weather)	Weather service	Search and booking content

Layer	Technology used	Planned in Chapter 2	Purpose in Voya
AI and ML	Own TF-IDF classifier trained with scikit-learn; OpenAI (chatbot); Gemini (content moderation)	OpenAI, scikit-learn recommender	AI Summarise, AI Travel Assistant, moderation
Documents	`pdf`, `printing`, `qr_flutter`, jsPDF, SheetJS	PDF and Excel export (4.4)	Booking documents, reports in PDF and Excel

Four technologies differ from the plan in Chapter 2. Google Maps and Google Places require a billing account, so OpenStreetMap was used for free place data, and the Google Routes API is used only for live traffic. Stripe replaced PayPal because its test mode and Flutter SDK made in-app payment simpler to build and test. A separate Flask server was not needed because the trained model was exported to JSON and runs inside a Node.js Cloud Function. The attraction recommender uses the weighted formula designed in Section 4.8.1, while scikit-learn was used to train the discussion-summary classifier.

5.1.2 Project structure

The Flutter code follows a layered MVC + Repository pattern, so that UI code never talks to Firestore directly. These folders correspond to the Presentation, Business Service and Data and Integration layers of the package diagram (Figure 4.86). The planned REST routes and middleware layer was not needed, because the app and web consoles call Firebase and the Cloud Functions directly through their SDKs.

Folder	Files	Responsibility
`lib/views/`	85	Screens and widgets, grouped by feature (auth, explore, group, expenses, vote, insurance, community, profile, etc.)
`lib/controllers/`	21	Screen state and user actions (e.g. `vote_controller.dart`, `expenses_controller.dart`)
`lib/models/`	13	Data classes with `fromDoc` / `toMap` conversion

Folder	Files	Responsibility
`lib/repositories/`	12	All Firestore reads and writes (e.g. `trip_repository.dart`, `insurance_repository.dart`)
`lib/services/`	27	External APIs and helpers (Stripe, Duffel, RollingGo, weather, PDF, recommendation, moderation)
`functions/`	—	Cloud Functions: `index.js`, `itinerary.js`, `optimise.js`, `discussion_summary/`, `planner/`
`admin_web/`	28 pages	`admin/` (16 pages) and `partner/` (12 pages) consoles
`ml/discussion_summary/`	—	Dataset building, training, evaluation and export of the model

5.1.3 Module implementation

The modules below follow the eight modules defined in Section 1.7.2 and designed in Chapter 4. The five algorithms from Section 4.8 are implemented inside the Itinerary Management, Travel Discovery and Expenses modules. Requirement IDs (FRxx) refer to Table 3.6.

(a) Account Management Module

`auth_service.dart` wraps Firebase Authentication. Travellers register and log in with email/password or Google (FR01, FR03); every method ends in `FirebaseAuth.signInWithCredential`, so each produces one record in `users/{uid}`. A Facebook sign-in was tried then removed, keeping only email/password and Google. Password reset sends a Firebase verification email (FR05). Users update their profile (FR06), send, accept and search friend requests in `friendRequests`, block users in `users/{uid}/blocked` (FR07) and log out (FR08). Insurance partners register on the partner console with company details for admin approval (FR02); supporting documents cannot be uploaded yet. A two-factor authentication setting is stored per user (`twoFactorEnabled`), but the second verification step at login is not yet enforced (see 7.4).

(b) Itinerary Management Module

A travel group is a document in `trips/{tripId}` holding `memberIds`, dates, destination and a day-by-day plan (FR09–FR11). The create-plan wizard (`create_plan_wizard_controller.dart`) collects destination, dates and members; members join through an invite code or QR code, and the owner can remove members (FR13). Each trip has sub-collections for `messages` (group discussion, FR14, FR16, FR18), `votes` and `expenses`. Members create, view, update and delete itinerary items manually, also during the trip (FR23–FR26, FR34), and every plan change notifies the other members.

Members who turn on location sharing appear on a live member map (FR32–FR33). Between two activities the plan suggests a transport mode based on distance (FR28), and a Live Conditions card shows the day's weather from Open-Meteo and a traffic-aware travel time from the Google Routes API (partly FR30).

Group Voting Algorithm (Section 4.8.5). Members create polls whose options come from the saved list of flights, hotels, attractions and restaurants (FR21), and results update live for all members (FR22). `castVote` follows 4.8.5: it runs inside a Firestore transaction, so two members voting at the same time cannot overwrite each other, and it rejects votes on missing or closed polls, by non-members, for missing options, and duplicate votes:

```
await FirebaseFirestore.instance.runTransaction((tx) async {
  final voteSnap = await tx.get(voteRef);
  final tripSnap = await tx.get(tripRef);
  if (!voteSnap.exists) throw StateError('This poll no longer exists.');
  final live = TripVote.fromDoc(voteSnap);
  if (live.isClosed) throw StateError('This poll is closed.');
  final memberIds = List<String>.from(tripSnap.data()?['memberIds'] ?? []);
  if (!memberIds.contains(uid)) throw StateError('You are not a member of
this trip.');
  // ... option check, duplicate-vote check, then tx.update(...)
});
```

AI-Summarised Itinerary Algorithm (Section 4.8.3). The `generateItinerary` Cloud Function implements the 12 steps of 4.8.3 (FR19–FR20). It runs only after the traveller agrees in a consent dialog (step 1), retrieves and filters the group discussion (steps 2–4), extracts places, days and times (step 5), builds a structured prompt (step 6) and submits it to the AI service (step 7). In Voya the AI service is the project's own trained model (5.1.5) rather than an external LLM, so no chat data leaves Firebase. The answer is parsed and

validated against the trip dates and known places (steps 8–9); if parsing fails it retries once with the most recent half of the chat (step 10). Suggestions are displayed and nothing is saved until the traveller confirms (steps 11–12).

Discussion summarisation. `summariseDiscussion` classifies each chat message (for example proposal, support, objection, decision, small talk) with a TF-IDF character n-gram model, scores message importance, links replies to topics and applies the rule "someone proposes it and nobody objects = agreed". This identifies the key decisions and conflicting opinions described in the Itinerary Management scope (1.7.2). The training pipeline is described in 5.1.5.

Optimised Itinerary Generation Algorithm (Section 4.8.2). The `optimiseItinerary` Cloud Function reorders each day without an external AI (FR27). Meals, sunrise/sunset slots, flights and hotel check-in stay fixed as anchors. Between anchors, a two-opt local search reorders places to maximise the utility from 4.8.2:

```
Utility = RecommendationScore - TravelTimePenalty - WaitingTimePenalty -
CostPenalty
```

Travel time comes from `maps_service.js`, which uses the haversine distance between coordinates, so it can later be swapped for a paid maps API without changing callers. The optimised day replaces the plan only after the traveller confirms it (steps 10–11).

(c) Travel Discovery Module

AI-Based Attraction Recommendation Algorithm (Section 4.8.1). `recommendation_service.dart` ranks attractions on the Explore page with the weighted score from 4.8.1:

```
Score(a) = 0.30P(a) + 0.25R(a) + 0.20D(a) + 0.15B(a) + 0.10T(a)
```

P is preference (how often the user saved or viewed this category), R is rating, D is distance, B is budget fit and T is whether the place is open at the planned time. Mandatory filters (budget, wheelchair access, open now) remove places before scoring, so a place that fails them is never shown, not just ranked low.

Attractions and restaurants are loaded from OpenStreetMap and cached in `catalog_attractions` and `catalog_restaurants` with coordinates, opening hours, rating, price and accessibility (FR43–FR44). Attractions can be filtered by category, price, open now and wheelchair access; rating and distance feed the ranking score instead of acting as filters (FR45, partly). Restaurants are filtered by cuisine and price. Travellers

save places to a trip (FR46) and can report a new attraction with a photo, which is stored as pending in attraction_reports (FR35); the admin console does not yet have a page to review these submissions (FR50). Names are translated for foreign destinations, and a nearby map shows places around the traveller.

The AI Travel Assistant (`attraction_chat_service.dart`) is a chatbot that uses the OpenAI Chat Completions API to answer questions and recommend real attractions from Voya's catalog (FR29, attractions only).

Flights and hotels. `duffel_api_service.dart` searches flights by route, date and passengers, with filters for airline, price and stops (FR40–FR41). `rollinggo_api_service.dart` searches hotels by destination, dates, guests and rooms, with real amenities and a price filter (FR36–FR37). Hotels can be saved into the itinerary before booking (FR39). Chapter 3 planned to redirect users to external booking sites (FR38, FR42); Voya instead completes the booking inside the app, so the whole trip stays in one place. Checkout calls the `createPaymentIntent` Cloud Function, so the Stripe secret key never leaves the server:

```
exports.createPaymentIntent = onCall({ secrets: [stripeSecretKey] }, async
(request) => {
  if (!request.auth) throw new HttpsError("unauthenticated", "Sign in before
paying.");
  const { amount, currency } = request.data || {};
  if (typeof amount !== "number" || amount <= 0)
    throw new HttpsError("invalid-argument", "amount must be a positive
number.");
  const stripe = new Stripe(stripeSecretKey.value());
  const pi = await stripe.paymentIntents.create({
    amount: Math.round(amount), currency: currency.toLowerCase(),
    metadata: { uid: request.auth.uid }, automatic_payment_methods: {
enabled: true },
  });
  return { clientSecret: pi.client_secret };
});
```

After payment, the booking is saved under `users/{uid}/bookings` and a PDF booking document with a QR code can be downloaded.

(d) Expenses Module

Shared-Expense Distribution Algorithm (Section 4.8.4). Travellers create, view, update and delete expenses by manual entry (FR51–FR55) and attach a receipt photo. Expenses can be split equally, per item or by custom amounts (FR58). Following 4.8.4, amounts are converted to integer cents, divided, and the leftover cents are given out one by one, so the shares always add up exactly to the total:

```
Map<String, double> splitExpenseEqual(double totalAmount, List<String>
memberUids) {
  _validateExpenseSplitInputs(totalAmount, memberUids.length);
  final totalCents = _expenseToCents(totalAmount);
  final n = memberUids.length;
  final baseShare = totalCents ~/ n;
  final remainder = totalCents % n;
  final shares = <String, double>{};
  for (var i = 0; i < n; i++) {
    shares[memberUids[i]] = _expenseFromCents(baseShare + (i < remainder ? 1
: 0));
  }
  return shares;
}
```

For example, RM10.00 split among 3 members gives RM3.34, RM3.33 and RM3.33. The Expenses tab shows how much you owe and how many bills are settled or pending (FR57, FR59), spending is shown as a donut chart, and the Traveller Detailed Expense Report (4.4.4) can be exported as a PDF.

(e) Community Module

Travellers publish posts with text, images, hashtags and a linked itinerary (FR61), browse a feed with recommended posts (FR63), and rate and review other users' posts (FR66–FR67). A Like button replaces the bookmark planned in FR69. Every post and review passes `content_moderation_service.dart` before it goes live; flagged content is blocked and logged in `moderation_log`. Travellers can report a post, and admins review reports and remove posts (FR70–FR71).

(f) Insurance Module

Insurance partners register on the partner console and are approved, rejected or suspended by the admin, and each decision notifies the partner (FR84, FR86). A rejected partner sees the reason on their Notifications page and is blocked after 30 days. A

suspended partner cannot publish plans or handle claims, but their existing plans stay visible to travellers. Approved partners create, view, update and delete plans in `insurance_plans` (FR79–FR82). Travellers browse, view and buy plans in the app (FR74–FR75; there is no search or comparison, FR72–FR73), which writes `insurance_transactions`, and submit claims and refunds with a reason and description (FR76–FR78); supporting documents cannot be attached yet. Partners review and approve or reject them (FR87–FR90), and admins oversee all transactions, claims and refunds (FR83). Partners export the Insurance Operations Report (4.4.2 B) as PDF or Excel.

(g) Emergency Module

For each trip, the emergency page shows the destination country's emergency numbers and the traveller's embassy from `country_emergency_info`, maintained by the admin (FR91–FR92). Each number can be called with one tap (FR93), and the traveller can share their current location with trusted contacts through the phone's share sheet (FR94).

(h) Administration Module

The admin web console (`admin_web/admin/`) lets administrators view traveller accounts and mark them as suspended (FR95–FR96; the mobile app does not yet block a suspended account), approve or suspend insurance partners (FR97), manage attractions and restaurants (FR47–FR49), moderate content and maintain country emergency information. The Reports page provides the Community Post Analysis, Insurance Operations and User Account reports from Section 4.4, plus a Platform Overview, and exports each as PDF or Excel (FR98).

5.1.4 Database design

Firestore is document-based, so data is grouped by how the app reads it. Trip data lives under the trip so one listener loads the chat, polls and expenses together.

Collection	Sub-collections	Main contents
`users`	`friends`, `blocked`, `notifications`, `bookings`	Profile, preferences, 2FA setting
`friendRequests`	—	Pending and accepted friend requests

Collection	Sub-collections	Main contents
`trips`	`messages`, `votes`, `expenses`	Members, dates, destination, day plans
`posts`	`reviews`, `viewers`	Community posts, hashtags, ratings and reviews on posts, view tracking
`catalog_attractions`, `catalog_restaurants`	—	Cached OpenStreetMap place data with lat/lng, rating, price, opening hours, accessibility
`catalog_hotels`, `catalog_flights`	—	Cached hotel (RollingGo) and flight (Duffel) results, favourites
`country_emergency_info`	`embassies`	Emergency numbers and embassy info
`insurance_partners`, `insurance_plans`, `insurance_transactions`, `claims`, `refunds`	—	Insurance operations
`admins`, `moderation_log`, `post_view_stats`, `app_cache`	—	Admin roles, moderation records, statistics, API cache

5.1.5 Machine learning model implementation

The discussion-summary classifier was trained on 16,892 messages from 1,444 chats: generated template chats, 44 hand-written chats (weighted ×5) and public datasets. Models were chosen by 5-fold GroupKFold cross-validation on training chats only. Naive Bayes, Logistic Regression and Linear SVM were compared on word and character TF-IDF features.

Model	CV macro-F1 (%)	test_unseen macro-F1 (%)	test_challenge accuracy (%)
Majority class (baseline)	4.4	6.9	52.9

Model	CV macro-F1 (%)	test_unseen macro-F1 (%)	test_challenge accuracy (%)
Naive Bayes + word	83.2	89.3	74.1
Linear SVM + char, C=0.3	92.4	97.6	78.2
LogReg + char, C=3, sqrt class weights (selected)	93.0	97.8	79.3

The selected model was exported to JSON and runs inside the Cloud Function with no external AI call. With the "no objection = agreed" rule, decision accuracy was 78.0% on the challenge set and 71.9% on a frozen second challenge set (the old majority rule scored 43.8% there).

5.1.6 Security implementation

Security follows the design in Section 4.5:

- **Authentication (4.5.1):** Firebase Authentication handles passwords and tokens; passwords are never stored in Firestore. Callable Cloud Functions reject requests without a signed-in user.
- **Role-based access control (4.5.2):** Firestore security rules check membership and roles. Only trip members read or write a trip's chat, votes and expenses; admins are listed in `admins/{uid}` and partners in `insurance_partners/{uid}`. A vote update may only change the `options` field.
- **Data protection (4.5.3):** All traffic uses HTTPS. The Stripe secret key is kept in Google Secret Manager and read only by Cloud Functions. Location sharing is off by default and can be turned off at any time.
- **Input validation (4.5.4):** Both client and server validate inputs, for example positive amounts, valid guest counts and expense shares that add up to the total. Posts and reviews pass content moderation before publishing.
- **Consent:** AI Summarise refuses any request without `consent: true` from the user.
- **Not yet as designed:** the 2FA second step, full audit logging (only moderation is logged) and keeping all third-party API keys on the server (the Duffel, RollingGo, OpenAI, Gemini and Google Routes keys are still inside the app). These are listed in 7.4.

5.2 Testing Strategies and Approaches

Following the plan in Section 1.6.1, Voya went through unit, integration, functional, performance and user acceptance testing. Group features such as votes and shared money were tested with several accounts at once, because their errors only appear when members act together.

Level	Approach	What was tested in Voya	Tools
Unit testing	White-box	The five algorithms of Section 4.8, the decision rule and the ML runtime	Node.js test scripts, Python scripts
Integration testing	Grey-box	App ↔ Firestore rules, App ↔ Cloud Functions, Stripe test payments, Duffel, RollingGo, OpenStreetMap and OpenAI APIs	Android emulator, Firebase Console, Stripe Dashboard
Functional (system) testing	Black-box	Every functional requirement in Table 3.6 that was implemented	Android phone and emulator, admin and partner web in Chrome
Performance testing	Black-box	Non-functional requirements 3.4.2 A: UI response within 2 s, AI itinerary within 30 s	Stopwatch timing on device, Cloud Functions execution time in Firebase Console
User acceptance testing (UAT)	Black-box	Ease of use and usefulness for real travel groups	Questionnaire (Likert scale 1–5)

Non-functional checks were also done: response time of Cloud Functions, security rules (a non-member trying to read a trip's chat must be denied), and behaviour when an external API fails (the app shows "No result" or "-" instead of fake data).

The machine learning model was evaluated separately with held-out test sets that were never used for training or model selection (Section 5.1.5).

5.3 Test Plan

Phase	Scope	Environment	Entry criteria	Exit criteria
1. Unit testing	Algorithms in `functions/` and `ml/`	Node.js 22, Python 3	Module code complete	All automated checks pass
2. Integration testing	Firestore, Cloud Functions, Stripe, external APIs	Firebase project `voya-f69f0`, Stripe test mode	Functions and rules deployed	Each integration returns correct data or a handled error
3. Functional testing	Test cases TC01–TC62	Android emulator + physical Android phone, Chrome for web consoles	Release build installed	All high-priority test cases pass
4. Performance testing	Test cases TC63–TC64	Physical Android phone on normal Wi-Fi	Functional testing passed	Response times within the limits in 3.4.2 A
5. UAT	Complete trip-planning flow	Testers' own phones	Functional testing passed	Average rating ≥ 4 out of 5

5.4 Test Data

Type	Test data used
User accounts	Traveller accounts (3–4 per test group), one admin account, one approved and one pending insurance partner
Valid inputs	Trip to Bangkok 3 days 2 nights, 4 members; expense RM10.00 split among 3; hotel booking 2 adults 1 room
Boundary values	Expense RM0.01 and RM0.00, 1-member trip, start date = end date, guest count 0 and maximum

Type	Test data used
Invalid inputs	Malformed email, empty required fields, password under 6 characters, negative amounts, custom shares not adding up to total
Payment	Stripe test card 4242 4242 4242 4242 (success), 4000 0000 0000 0002 (declined)
Moderation	Posts and reviews containing profanity in English, Chinese and Malay, plus clean text
Chat data	Scripted group chats (e.g. "Let's go Petronas Twin Towers on day 2" / "ok" / "skip Disneyland") for AI Summarise
ML data	16,892 training messages from 1,444 chats; separate test sets test_seen (1,699), test_unseen (1,414), test_challenge (174), test_challenge2 (140), test_smalltalk (126), test_public (2,312), test_offtopic (505)

5.5 Test Cases

5.5.1 Automated test results

These scripts were run on the development machine on 25 Oct 2026.

Test script	What it checks	Result
`test_optimise.js`	8 optimiser scenarios, incl. 2-opt reordering places by real distance, anchors kept fixed	All checks passed
`test_planner.js`	AI Summarise planner: day/time extraction, Voya picks for wishes without place names	All checks passed
`test_osm_places.js`	Finding unsaved places on OpenStreetMap near the trip	All checks passed
`test_parity.py`	Python model and JavaScript runtime give identical output	470 / 470 chats identical

Test script	What it checks	Result
`test_functions_mock.js`	Runs the Cloud Functions against a fake Firestore	Failed: the test does not send `consent: true`, which the consent gate now requires. Test needs updating, not the function

5.5.2 Functional test cases

Version 1 (summary format) is below: one row per test case. Version 2 (step-by-step format, one row per step) is in the tab `Test Cases – Version 2 (Step-by-Step)`. Fill in the Actual Result and Status columns after running each case on the release build.

TC ID	Module	Test description	Test data / steps	Expected result	Actual result	Status
TC01	Authentication	Register with valid details	Name, valid email, password "Test1234"	Account created, user goes to onboarding		
TC02	Authentication	Register with invalid email	"abc@"	Error "Enter a valid email", no account created		
TC03	Authentication	Login with wrong password	Registered email, wrong password	Error message, user stays on login page		
TC04	Authentication	Google sign-in	Tap Google, choose account	User logged in, profile created in `users`		
TC05	Authentication	Two-factor authentication	Enable 2FA, log out, log in	Verification step required before home page		
TC06	Trip planning	Create a group trip	Destination Bangkok, 3 days, add 3 friends	Trip appears on Home for all 4 members		
TC07	Trip planning	End date before start date	Start 10 Dec, end 8 Dec	End date cannot be set before the start date (it resets to the start date)		
TC08	Trip planning	Edit plan and leave without saving	Change an activity, press back	Draft-save alert shown		

TC ID	Module	Test description	Test data / steps	Expected result	Actual result	Status
TC09	Trip planning	Plan update notification	Member A edits the day plan	Other members receive a notification		
TC10	Trip planning	Activity schedule conflict	Add an activity for User B at a time B already has one, then open AI Summarise	Schedule conflict card lists the clash		
TC11	Group chat	Send message and attachment	Text + image	All members see it in real time		
TC12	Voting	Create poll from saved list	Pick 3 saved places	Poll with 3 options visible to members		
TC13	Voting	Vote once	Choose option 1	Count of option 1 increases by 1		
TC14	Voting	Duplicate vote (single choice)	Vote again on the same poll	Rejected, SnackBar message shown		
TC15	Voting	Vote on closed poll	Poll past deadline	Rejected "This poll is closed"		
TC16	Expenses	Equal split with remainder	RM10.00 among 3 members	Shares RM3.34, RM3.33, RM3.33 (total RM10.00)		
TC17	Expenses	Custom split not matching total	RM100, shares 40 + 40	Error, expense not saved		
TC18	Expenses	Negative or zero amount	RM-5 / RM0	Validation error		
TC19	Expenses	Mark as settled	Member marks share paid	Status shows settled, balance updated		
TC20	Expenses	Attach and edit receipt	Upload photo, then replace	New receipt shown on the expense		
TC21	Explore	Recommendation ranking	Open Explore for Kuala Lumpur	Attractions ordered by score, closed places hidden when "Open now" is on		
TC22	Explore	Wheelchair filter	Turn on "Wheelchair accessible"	Only accessible attractions shown		
TC23	Explore	No results	Search an unknown place	"No result" message, no error box		

TC ID	Module	Test description	Test data / steps	Expected result	Actual result	Status
TC24	Nearby	Nearby map	Allow location	Pins of places near the user		
TC25	AI Summarise	Consent required	Tap AI Summarise	Consent dialog shown before reading chat		
TC26	AI Summarise	Agreed place added	Chat: "Petronas Towers day 2?" / "ok"	Suggestion: add Petronas Towers on Day 2		
TC27	AI Summarise	Rejected place	Chat: "skip Disneyland"	Suggestion: remove Disneyland		
TC28	AI Summarise	Nothing auto-saved	Close sheet without applying	Plan unchanged		
TC29	Optimise	Optimise a day	Day with 4 places in random order	Places reordered to shorten travel, meals stay at meal times		
TC30	Flights	Search flights	KUL → BKK, 1 adult, date	Duffel results with price and times		
TC31	Hotels	Invalid guest count	0 adults	Validation error		
TC32	Payment	Successful payment	Card 4242 4242 4242 4242	Booking saved, PDF with QR available		
TC33	Payment	Declined card	Card 4000 0000 0000 0002	Payment failed message, no booking saved		
TC34	Insurance	Buy a plan	Choose plan, fill traveller info, pay	Transaction appears in app and partner console		
TC35	Insurance	Partner registration rejected	Admin rejects partner	Partner sees the rejection reason on the Notifications page; account blocked after 30 days		
TC36	Insurance	Partner suspended	Admin suspends an approved partner	Partner console shows Suspended; partner cannot publish plans or handle claims		
TC37	Community	Create post with hashtags	Title, image, #bangkok	Post visible in feed and hashtag search		

TC ID	Module	Test description	Test data / steps	Expected result	Actual result	Status
TC38	Moderation	Post with profanity	Offensive word in title	Post blocked, entry in `moderation_log`		
TC39	Moderation	Post review with profanity	Offensive review on a community post	Review blocked		
TC40	Friends	Send and accept friend request	User A → User B	Request in B's pending section, both become friends		
TC41	Friends	Block user	A blocks B	B appears in A's Blocked Users list		
TC42	Safety	Emergency info	Open trip to Thailand	Emergency numbers and embassy shown		
TC43	Security	Non-member reads trip	Use another account's trip ID	Permission denied by Firestore rules		
TC44	Admin web	Admin login	Admin email + password	Overview dashboard shown		
TC45	Admin web	Non-admin login	Traveller account	Access denied		
TC46	Admin web	Edit country emergency info	Change police number	App shows the new number		
TC47	Admin web	Export report	Reports → Export PDF	PDF with Voya branding downloaded		
TC48	Partner web	Process claim	Approve a claim	Claim status updated, traveller notified		

5.5.3 Additional Test Cases (Remaining Requirements and Non-Functional Requirements)

These cases cover implemented requirements that TC01–TC48 do not, and the non-functional requirements in Section 3.4.2.

TC ID	Module (FR / NFR)	Test description	Test data / steps	Expected result	Actual result	Status
TC49	Account (FR05)	Reset password	Login → Forgot password, enter registered email	Reset email received; new password works		

TC ID	Module (FR / NFR)	Test description	Test data / steps	Expected result	Actual result	Status
TC50	Account (FR08)	Log out	Profile → Log out	Returned to login page; protected pages need login again		
TC51	Itinerary (FR13)	Owner removes a member	Owner opens Group Info, removes User D	User D no longer sees the trip		
TC52	Itinerary (FR32–FR33)	Live member location	User B turns on location sharing, User A opens member map	User B shown on map; disappears after B turns sharing off		
TC53	Itinerary (FR28)	Transport suggestion	Day with two activities 3 km apart	Suggested transport mode shown between the two activities		
TC54	Itinerary (FR30)	Live conditions	Open today's plan for a trip in Kuala Lumpur	Weather and traffic-aware travel time shown		
TC55	Travel Discovery (FR29)	AI Travel Assistant	Explore → Attractions → Ask AI: "family-friendly places in KL"	Reply recommends real attractions from the catalog		
TC56	Travel Discovery (FR35)	Report a new attraction	Profile → Report New Attraction, enter name, location and photo	Submission saved with status pending		
TC57	Travel Discovery (FR39)	Save hotel to itinerary	Hotel page → Add to plan	Hotel appears in the trip plan before booking		
TC58	Expenses (FR58)	Per-item split	Bill with 3 items assigned to different members	Each member's share = sum of their items		
TC59	Emergency (FR93)	One-tap emergency call	Trip to Thailand → Emergency → tap police number	Phone dialer opens with the number		
TC60	Emergency (FR94)	Share location	Emergency → Shared Location	Share sheet opens with current location link		

TC ID	Module (FR / NFR)	Test description	Test data / steps	Expected result	Actual result	Status
TC61	Administration (FR96)	Suspend traveller	Admin → Users → set status Suspended	Traveller can no longer use the app		
TC62	Insurance (FR88)	Partner approves refund	Partner → Refunds → approve a pending refund	Status Approved; traveller sees the update		
TC63	Performance (NFR A-i)	UI response time	Open Home, Explore, Plan, Profile; time each	Each screen responds within 2 s		
TC64	Performance (NFR A-ii)	AI itinerary time	Run AI Summarise on a 50-message chat; time it	Suggestions shown within 30 s		
TC65	Availability (NFR B-ii)	Reconnect after network loss	Turn off Wi-Fi, add an itinerary item, turn Wi-Fi on	Item syncs to other members after reconnect		
TC66	Reliability (NFR C-ii)	Shared data consistency	Two members edit the same day at the same time	Both see the same final plan, no duplicates		
TC67	Security (NFR D-iii)	Role restriction	Insurance partner opens an admin page URL	Access denied		
TC68	Usability (NFR E-ii)	Navigation depth	From Home, reach Expenses, Explore, Community and Profile	Each reached within 3 taps		

5.5.4 Requirements Traceability

Module (Section 1.7.2)	Requirements (Table 3.6)	Test cases
Account Management	FR01–FR08	TC01–TC05, TC40–TC41, TC49–TC50
Itinerary Management	FR09–FR35	TC06–TC15, TC25–TC29, TC51–TC54
Travel Discovery	FR36–FR50	TC21–TC24, TC30–TC31, TC55–TC57
Expenses	FR51–FR60	TC16–TC20, TC58

Module (Section 1.7.2)	Requirements (Table 3.6)	Test cases
Community	FR61–FR71	TC37–TC39
Insurance	FR72–FR90	TC32–TC36, TC48, TC62
Emergency	FR91–FR94	TC42, TC59–TC60
Administration	FR95–FR98	TC44–TC47, TC61
Non-functional (Section 3.4.2)	Performance, availability, reliability, security, usability	TC43, TC45, TC63–TC68

Requirements that are not implemented (listed in 7.4) have no test cases. TC05 (2FA step, FR04) and TC61 (suspended traveller blocked, FR96) test what the requirements ask for and are expected to fail until those parts are built.

5.5.5 User Acceptance Testing

Ask 5–10 testers to plan one trip together and then rate these statements from 1 (strongly disagree) to 5 (strongly agree):

1. The app is easy to learn.
2. Creating a group trip and inviting friends is simple.
3. Voting helps the group decide faster.
4. The expense split is clear and correct.
5. AI Summarise gives useful plan suggestions from our chat.
6. Optimise gives a sensible order for the day.
7. I would use Voya for a real group trip.

5.6 Chapter Summary and Evaluation

All eight modules from Section 1.7.2 were implemented, and the five algorithms follow the designs in Section 4.8. Of the 98 functional requirements, 67 are fully implemented, 15 partly, 3 in a different way from the design, and 13 are not yet implemented (listed in 7.4). All automated algorithm tests passed, and the model runtime gave identical results to the trained Python model on 470 out of 470 chats. One automated test (`test_functions_mock.js`) is out of date because of the new consent check and should be updated. After running TC01–TC68 and the UAT, add the totals here: cases passed out of 68, defects found and fixed, measured response times and the average UAT rating.

Chapter 6: System Deployment

Voya is deployed as an Android app connected to the Firebase project `voya-f69f0`, with four Cloud Functions on Node.js 22 and a static admin/partner web console. This chapter lists the environment, the deployment steps and the problems met during deployment.

6.1 Deployment Environment

Component	Environment
Mobile app	Android APK, package `my.fyp.voya`, built with Flutter (Dart SDK ^3.10)
Backend project	Firebase project `voya-f69f0` (Blaze plan, required for Cloud Functions and Secret Manager)
Authentication	Firebase Authentication: Email/Password, Google
Database and files	Cloud Firestore, Firebase Storage, protected by `firestore.rules` and `storage.rules`
Server functions	Cloud Functions v2, Node.js 22: `createPaymentIntent`, `generateItinerary` (120 s, 512 MiB), `optimiseItinerary` (60 s, 256 MiB), `summariseDiscussion`
Secrets	Google Secret Manager (Stripe secret key)
Web consoles	`admin_web/` static HTML/JS pages using the Firebase Web SDK, opened in Chrome
Third-party services	Stripe (test mode), Duffel, RollingGo, OpenStreetMap / Nominatim, Open-Meteo
Development machine	Windows PC, Android Studio, Flutter SDK, Node.js, Firebase CLI

The deployed system is simpler than the deployment diagram in Figure 4.85. Firebase Cloud Messaging push notifications were replaced by in-app notifications stored in Firestore. The separate traveller and partner application servers, API gateway and firewall

nodes are covered by Firebase's managed services, since the clients call Firebase and Cloud Functions directly over HTTPS. The admin and partner consoles currently run as static pages opened in a browser and are not yet published on Firebase Hosting.

6.2 Deployment Process

1. **Create the Firebase project.** Create `voya-f69f0` in the Firebase Console, upgrade to the Blaze plan, and enable Authentication, Firestore and Storage.
2. **Enable sign-in providers.** In Authentication → Sign-in method, enable Email/Password, Google.
3. **Register the Android app.** Add package `my.fyp.voya`, add the SHA-1 and SHA-256 fingerprints of the signing key, then download `google-services.json` into `android/app/`.
4. **Connect Flutter to Firebase.** Run `flutterfire configure`, which writes `lib/firebase_options.dart`.
5. **Store secrets.** Run `firebase functions:secrets:set STRIPE_SECRET_KEY` so the key is held only in Secret Manager.
6. **Install function dependencies.** In `functions/`, run `npm install`.
7. **Deploy security rules.** Run `firebase deploy --only firestore:rules,storage`.
8. **Deploy Cloud Functions.** Run `firebase deploy --only functions`, then check in the Firebase Console that all four functions are active.
9. **Build the Android app.** Create a release keystore, set it in `android/key.properties`, add its SHA-1 to Firebase (step 3), then run `flutter build apk --release`. The APK is in `build/app/outputs/flutter-apk/app-release.apk`.
10. **Install and smoke-test.** Install the APK on a phone and run the high-priority cases from 5.5.2 (login, create trip, pay with a test card).
11. **Set up the admin console.** Open `admin_web/setup-admin.html` once to create the first admin account, then remove or restrict that page. Admins log in at `admin/login.html`; partners register at `partner/register.html`.
12. **Seed data.** Use "Seed demo data" in the admin or partner console to fill plans, claims and transactions for the demonstration.

6.3 Issues and Solutions

Issue	Cause	Solution
`firebase deploy --only functions` failed with "User code failed to load. Timeout after 10000"	Loading `firebase-functions` and `stripe` took about 45 s because Windows Defender scanned every file in `node_modules`; Firebase allows only 10 s	Added an antivirus exclusion for the project folder, then deployed again
Google sign-in failed on Android	No SHA-1 fingerprint registered, so `oauth_client` in `google-services.json` was empty	Added debug and release SHA-1 in Firebase, downloaded a new `google-services.json`, ran `flutter clean`
AI Summarise returned "getting a lot of requests right now"	The free Gemini API was rate-limited	Replaced Gemini with Voya's own trained model and rule-based planner, so the feature no longer depends on an external AI quota
AI Summarise returned 403	The new function did not yet allow invocation by signed-in users	Granted the Cloud Run invoker permission for the function and redeployed
Notifications showed "permission denied"	New rules for `users/{uid}/notifications` were written but not deployed	Ran `firebase deploy --only firestore:rules`
Admin report queries across all trips failed	Rules did not cover `collectionGroup` queries on `expenses` and `reviews`	Added `match /{path=**}/...` rules for those collection groups
Optimise ignored real distances	Catalog places were saved without `lat` / `lng`	OSM refresh now writes coordinates; old cached places update on the next refresh
Function test failed after an update	`test_functions_mock.js` does not send the new `consent: true` flag	Update the test to send consent

6.4 Chapter Summary and Evaluation

Voya runs fully on managed Firebase services, so no server has to be maintained, and the Stripe secret key stays on the server side. Most deployment problems came from configuration (fingerprints, rules not deployed, permissions, antivirus) rather than from the code, and each was fixed as described above. Compared with Figure 4.85, three items remain: publishing the web consoles on Firebase Hosting, moving the remaining third-party API keys from the app into Cloud Functions, and switching Stripe from test mode to live mode. The app has also been built and tested only for Android so far.

Chapter 7: Discussions and Conclusion

7.1 Summary

Voya, an AI-Driven Contextual Travel Management system, was built to solve the three problems stated in Section 1.3: a fragmented planning process, the difficulty of planning as a group, and a lack of personalisation. Groups create a shared trip, discuss, vote and split expenses in real time. Voya turns the group's discussion into itinerary suggestions (AI Summarise), reorders each day to cut travel time, recommends attractions and answers questions through an AI assistant. Travellers can book flights and hotels, buy insurance, share posts and view emergency information, while administrators and insurance partners manage the platform through a web console.

7.2 Achievements

Objective (Section 1.1)	Achieved through	Evidence
To develop a collaborative group travel planning system that enables travellers to coordinate itineraries, share preferences, make collective decisions, and manage shared travel expenses within a unified platform	Shared trips with real-time chat and day plans; transaction-safe group voting (4.8.5); cents-accurate expense splitting (4.8.4); live member location	TC06–TC20, TC51–TC52, TC58

Objective (Section 1.1)	Achieved through	Evidence
To improve the overall travel experience by integrating AI-powered assistance features, including chatbot support and discussion summarisation, to provide personalised recommendations, planning insights, and itinerary suggestions	AI Travel Assistant chatbot; own discussion-summary model; AI-Summarised Itinerary (4.8.3); weighted attraction recommendation (4.8.1); optimised itinerary (4.8.2)	Model results in 5.1.5, TC21, TC25–TC29, TC55
To enhance the efficiency and convenience of travel planning by integrating multiple travel management functions into a single platform	Flight and hotel search and in-app booking, insurance, community, emergency information and admin/partner consoles in one system	TC30–TC48, TC56–TC62

All three objectives were met. At requirement level, 67 of the 98 functional requirements in Table 3.6 are fully implemented and 15 partly; the rest are listed in 7.4.

7.3 Contributions

- **Centralised travel planning platform (1.5.1):** planning, booking, expenses, insurance, community and emergency information are in one app instead of several chat groups, spreadsheets and booking sites.
- **Simplified itinerary generation (1.5.2):** AI Summarise turns the group's chat into plan suggestions, and the optimiser orders each day automatically.
- **Improved group travel collaboration (1.5.3):** real-time discussion, fair voting, shared expenses and member locations keep the group in sync.
- **Intelligent travel recommendations (1.5.4):** weighted attraction ranking with mandatory filters, and an AI assistant for questions.
- **Technical contributions:** a self-trained model that reads multilingual group chat (English, Chinese, Malay) and decides what the group agreed without a paid AI service; a two-opt optimiser with fixed anchors such as meals and flights; and a rounding-safe expense algorithm that always splits to the exact total.
- **To the developer:** experience in full-stack mobile development, cloud security rules, payment integration, and building and evaluating a machine learning model end to end.

7.4 Limitations and Future Improvements

Limitation	Future improvement
2FA setting is saved but no second verification step at login (FR04)	Add a code by email or SMS after password login, required for admin and partner accounts
Missing group and content controls: delete a travel group (FR12), edit or delete discussion messages (FR15, FR17), edit or delete own posts (FR62, FR64), remove a friend (FR07)	Add these actions; Firestore rules already allow the owner to delete a trip
No OCR receipt scanning (FR56) or payment reminders (FR60)	Add on-device text recognition for receipts and scheduled reminder notifications
No import of itineraries from community posts (FR31, FR68), no AI summary when sharing (FR65), no video posts (FR61)	Copy a post's linked plan into a new trip; reuse the discussion model to summarise a plan
No insurance plan search or comparison (FR72, FR73); admin cannot disable a single plan (FR85); partner registration and claims cannot carry documents (FR02, FR77); a suspended partner's plans stay visible	Search and side-by-side comparison; per-plan status switch; file upload to Firebase Storage; hide plans of suspended partners
Itinerary is not adjusted automatically for weather, traffic or closures (FR30)	Use the Live Conditions data to suggest moving outdoor activities
Hotel room type, hotel facility filter, flight cabin class and duration filter missing (FR36, FR37, FR40, FR41)	Pass these options to the RollingGo and Duffel APIs
AI chatbot covers attractions only (FR29); emergency contacts use the trip country, not GPS (FR91); no admin page to review reported attractions (FR50); suspended travellers can still use the app (FR96).	Extend the chatbot to restaurants and hotels; detect the current country from GPS; add a review page for attraction_reports; check account status at login.

Limitation	Future improvement
Third-party API keys (Duffel, RollingGo, OpenAI, Gemini, Google Routes) are inside the app, unlike Section 4.5.3; audit log covers moderation only (4.5.5)	Move these calls into Cloud Functions; log admin and partner actions
Stripe in test mode; Android only; web consoles not hosted	Go live with Stripe; build for iOS; publish consoles on Firebase Hosting
Discussion model is weaker on new kinds of chats (71.9% decision accuracy on the frozen challenge set)	Label more real group chats and retrain

7.5 Issues and Solutions

These are project-level issues; deployment problems are in 6.3.

Issue	Solution
Google Maps and Google Places, planned in Chapter 2, need a billing account	Used free OpenStreetMap data and Nominatim geocoding; kept the Google Routes API only for live traffic
Free AI APIs were rate-limited and paid ones were outside the budget	Built and trained Voya's own discussion model, so AI Summarise costs nothing to run; OpenAI is used only for the chatbot
No real labelled group-chat data existed	Created template chats, hand-wrote 44 chats, added public datasets and kept separate frozen test sets
Redirecting to external booking sites (FR38, FR42) would split the trip across several apps	Built in-app booking with Duffel, RollingGo and Stripe instead of PayPal
Free place data from OpenStreetMap often had missing photos, names or descriptions	Added fallbacks (Wikipedia, website images) and admin tools to fix names and descriptions
Many features for one developer in limited time	Followed the incremental model (1.6.1), built module by module, and re-tested after each change

Issue	Solution
Algorithms in the code drifted from the design in Section 4.8	Audited all five algorithms against the design and updated the code to match

7.6 Conclusion

Voya meets its three objectives by combining collaboration, AI assistance and travel services in one platform. Its main strength is that the group's own discussion drives the itinerary, through a model built and evaluated for this project. The remaining gaps against the requirements, mainly the 2FA step, OCR, itinerary import and moving API keys to the server, are clear next steps for turning Voya into a production app.